Maximum Subarray Sum Problem: A Comparative Study of Algorithm Design Paradigms

M. Faraz Ansari (29201)
Fatima Faisal (28989)
Marium Ali Lakhani (29183)
Khadeja Qureshi (27200)
Areesha Faseeh (29007)

Spring 2026

Abstract

The Maximum Subarray Sum problem seeks the contiguous subarray with the largest sum within a given array of integers. This paper presents a comprehensive comparative analysis of five algorithmic approaches: Brute Force ($O(n^3)$), Better Brute Force ($O(n^2)$), Prefix Sum ($O(n^2)$), Divide and Conquer ($O(n \log n)$), and Kadane's Algorithm ($O(n)$). Each algorithm was implemented in C++ and tested on systematically generated datasets of varying sizes (100 to 3000 elements) across random, all-negative, and all-positive distributions. Empirical results demonstrate that Kadane's Algorithm outperforms the Brute Force approach by a factor of approximately 29,000x at n=3000, highlighting the profound impact of algorithm design on computational efficiency. All implementations were verified to produce identical results, ensuring fair comparison. Theoretical operation count analysis confirms that the observed performance differences align precisely with asymptotic complexity expectations.

Contents

1 Introduction

1.1 Problem Description

The Maximum Subarray Sum problem is one of the most fundamental and well-studied problems in algorithm design. Given an array A of n integers (which may contain both positive and negative values), the objective is to find a contiguous subarray whose elements sum to the largest possible value.

1.2 Motivation and Real-World Relevance

Despite its simple formulation, the Maximum Subarray Sum problem appears frequently across diverse domains:

- **Financial Analysis:** Identifying the period of maximum profit in stock price data
- **Signal Processing:** Detecting the strongest signal segment within noisy waveforms
- **Bioinformatics:** Finding biologically significant segments in DNA sequences
- **Image Processing:** Identifying the brightest rectangular region in 2D images
- **Data Mining:** Locating periods of maximum activity in time-series data

1.3 Formal Problem Formulation

Let $A = [a_1, a_2, \ldots, a_n]$ be an array of n integers. A contiguous subarray is defined by indices i and j where $1 \leq i \leq j \leq n$, representing the elements $a_i, a_{i+1}, \ldots, a_j$. The sum of this subarray is given by:

$$S(i, j) = \sum_{k=i}^{j} a_k$$

The Maximum Subarray Sum problem seeks to find:

$$\text{max\_sum} = \max_{1 \leq i \leq j \leq n} S(i, j)$$

Additionally, the problem may optionally ask to return the starting and ending indices i and j where this maximum occurs.

1.4 Input and Output Specifications

Input:

- An array A of n integers, where $n \geq 1$
- Integers may be positive, negative, or zero
- No constraints on the range of values (practical limits depend on implementation)

Output:

- A single integer representing the maximum possible sum of any contiguous subarray

- Optionally, the starting and ending indices of that subarray

Example:

- **Input:** $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$
- **Output:** 6 (from subarray $[4, -1, 2, 1]$)

1.5 GitHub Repository

All code, datasets, and results are available at:

https://github.com/farazz1/Maximum-Subarray-DAA-Project-2026

2 Algorithm Descriptions

2.1 Brute Force — O(n^3)

The brute force approach exhaustively evaluates every possible contiguous subarray using three nested loops. The outer two loops fix the starting and ending indices, while the innermost loop computes the sum of elements between those indices. This approach recomputes each subarray sum from scratch, resulting in O(n^3) complexity.

Pseudocode:

```
maxSum = -infinity
for i = 0 to n-1:
    for j = i to n-1:
        currentSum = 0
        for k = i to j:
            currentSum += A[k]
        maxSum = max(maxSum, currentSum)
return maxSum
```

2.2 Better Brute Force — O(n^2)

The improved brute force eliminates the innermost loop by maintaining a running sum as the right boundary extends. Starting from each left index i, it incrementally adds elements as j increases, tracking the maximum encountered sum. This reduces complexity to O(n^2).

Pseudocode:

```
maxSum = -infinity
for i = 0 to n-1:
    currentSum = 0
    for j = i to n-1:
        currentSum += A[j]
        maxSum = max(maxSum, currentSum)
return maxSum
```

2.3 Prefix Sum — O(n^2)

The prefix sum approach preprocesses the array to build a cumulative sum array where $prefix[i]$ stores the sum of elements from index 0 to $i-1$. Any subarray sum can then be computed in O(1) as $prefix[j+1] - prefix[i]$, though two nested loops still evaluate all O(n^2) possible subarrays.

Pseudocode:

```
prefix[0] = 0
for i = 1 to n:
    prefix[i] = prefix[i-1] + A[i-1]
maxSum = -infinity
for i = 0 to n-1:
    for j = i to n-1:
        currentSum = prefix[j+1] - prefix[i]
        maxSum = max(maxSum, currentSum)
return maxSum
```

2.4 Divide and Conquer — O(n log n)

The divide and conquer approach recursively splits the array in half. The maximum subarray must lie entirely in the left half, entirely in the right half, or cross the midpoint. The crossing case is evaluated by a linear scan expanding outward from the midpoint. The recurrence $T(n) = 2T(n/2) + O(n)$ solves to O(n log n).

Pseudocode:

```
function maxCrossingSum(A, left, mid, right):
    leftSum = -infinity, sum = 0
    for i = mid down to left:
        sum += A[i]
        leftSum = max(leftSum, sum)
    rightSum = -infinity, sum = 0
    for i = mid+1 to right:
        sum += A[i]
        rightSum = max(rightSum, sum)
    return leftSum + rightSum

function maxSubarraySum(A, left, right):
    if left == right: return A[left]
    mid = (left + right) / 2
    return max(maxSubarraySum(A, left, mid),
               maxSubarraySum(A, mid+1, right),
               maxCrossingSum(A, left, mid, right))
```

2.5 Kadane's Algorithm — O(n)

Kadane's algorithm is the optimal dynamic programming solution. It maintains two variables: the maximum sum ending at the current position (*current*) and the overall maximum seen so far (*best*). At each element, it decides whether to extend the existing subarray or start a new one:

$$current = \max(A[i], current + A[i])$$

$$best = \max(best, current)$$

This single pass achieves O(n) time and O(1) space.

Pseudocode:

```
currentSum = A[0]
maxSum = A[0]
for i = 1 to n-1:
    currentSum = max(A[i], currentSum + A[i])
    maxSum = max(maxSum, currentSum)
return maxSum
```

3 Theoretical Analysis

Table 1 presents the complete theoretical complexity analysis for all five algorithms, including best-case, average-case, and worst-case time complexities.

Table 1: Complete Theoretical Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Space
Brute Force (All Subarrays)	$O(n^3)$	$O(n^3)$	$O(n^3)$	O(1)
Better Brute Force	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Prefix Sum	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(n)
Divide & Conquer	O(n log n)	O(n log n)	O(n log n)	O(log n)
Kadane's Algorithm	O(n)	O(n)	O(n)	O(1)

3.1 Analysis of Complexity Differences

Brute Force (All Subarrays): All three cases are $O(n^3)$ because the algorithm unconditionally evaluates every possible subarray using three nested loops, regardless of input characteristics. There is no early termination or optimization.

Better Brute Force: Similarly, this algorithm always examines all $O(n^2)$ subarrays. The removal of the innermost loop reduces constant factors but does not change the asymptotic complexity across cases.

Prefix Sum: The preprocessing step is always O(n), followed by $O(n^2)$ subarray evaluations. The space complexity increases to O(n) due to the prefix sum array storage.

Divide and Conquer: The recurrence $T(n) = 2T(n/2) + O(n)$ holds for all inputs, resulting in consistent O(n log n) performance. The recursion stack requires O(log n) additional space.

Kadane's Algorithm: A single linear pass guarantees O(n) time in all cases, with O(1) auxiliary space, making it theoretically optimal.

4 Implementation Details

4.1 Machine Specification

- **Operating System:** Windows 11
- **Processor:** Intel Core i-series

- **RAM:** 8 GB
- **Compiler:** g++ (MinGW/MSYS2, GCC 14.2.0)
- **Compiler Flag:** -O0 (optimizations disabled)
- **Language:** C++17

4.2 Programming Language and Libraries

All algorithms were implemented in C++ using the C++17 standard. The following standard library components were used:

- `<vector>` for dynamic arrays
- `<chrono>` for high-resolution timing
- `<fstream>` for dataset file I/O
- `<random>` with mt19937 for reproducible random number generation
- `<algorithm>` for min/max operations

4.3 Shared Test Harness

A unified test harness was developed so that all five algorithms could be benchmarked identically. Each algorithm was implemented in its own header file conforming to a common interface. The harness performs a correctness check on a known sample input before running any benchmarks, ensuring all algorithms produce identical answers. Results are written to both the terminal and a results.csv file.

4.4 Dataset Generation

Three dataset types were generated for each input size using fixed random seeds to ensure reproducibility across machines:

- **Seed 42:** Random datasets (values in $[-1000, 1000]$)
- **Seed 99:** All-negative datasets (values in $[-1000, -1]$)
- **Seed 7:** All-positive datasets (values in $[1, 1000]$)

Table 2: Dataset Configuration

Input Size (n)	Dataset Types	Total Datasets
100	Random, Negative, Positive	3
500	Random, Negative, Positive	3
1000	Random, Negative, Positive	3
2000	Random, Negative, Positive	3
3000	Random, Negative, Positive	3

4.5 Assignment of Algorithms

Table 3: Algorithm Assignment by Team Member

Member	ERP	Algorithm	Complexity
Areesha Faseeh	29007	Brute Force (All Subarrays)	$O(n^3)$
Marium Ali Lakhani	29183	Better Brute Force	$O(n^2)$
Khadeja Qureshi	27200	Prefix Sum	$O(n^2)$
Fatima Faisal	28989	Divide & Conquer	O(n log n)
Faraz Ansari	29201	Kadane's Algorithm	O(n)

5 Results and Discussion

5.1 Runtime Results — Random Dataset

Table 4 presents the runtime measurements for the random dataset.

Table 4: Runtime Results — Random Dataset (ms)

n	Brute Force	Better Brute	Prefix Sum	Divide & Conquer	Kadane
100	0.7492	0.0422	0.0494	0.0295	0.0032
500	86.4219	1.1261	0.8676	0.0599	0.0111
1000	627.2828	3.7828	3.2713	0.1188	0.0171
2000	3644.7458	10.8360	12.9106	0.2259	0.0254
3000	13766.6189	31.7550	30.7285	0.4700	0.0254

5.2 Runtime Results — Negative Dataset

Table 5 shows performance on all-negative arrays, where all algorithms correctly return the least-negative value.

Table 5: Runtime Results — Negative Dataset (ms)

n	Brute Force	Better Brute	Prefix Sum	Divide & Conquer	Kadane
100	0.7884	0.0412	0.3519	0.0107	0.0031
500	102.1067	1.1147	1.1441	0.0823	0.2012
1000	556.3389	2.6022	3.0778	0.0724	0.0144
2000	2915.5300	8.8907	10.4910	0.1108	0.0162
3000	15539.7629	30.0888	26.2112	0.2965	0.0684

5.3 Runtime Results — Positive Dataset

Table 6 presents performance on all-positive arrays, where the maximum subarray is always the entire array.

Table 6: Runtime Results — Positive Dataset (ms)

n	Brute Force	Better Brute	Prefix Sum	Divide & Conquer	Kadane
100	0.7799	0.0467	0.1012	0.0091	0.0032
500	97.7292	1.0680	1.2757	0.0483	0.0108
1000	553.4703	2.7052	2.4929	0.0863	0.0143
2000	2478.1930	7.8040	8.2813	0.2453	0.0211
3000	14440.8591	19.2636	15.8529	0.1868	0.0290

5.4 Visual Analysis

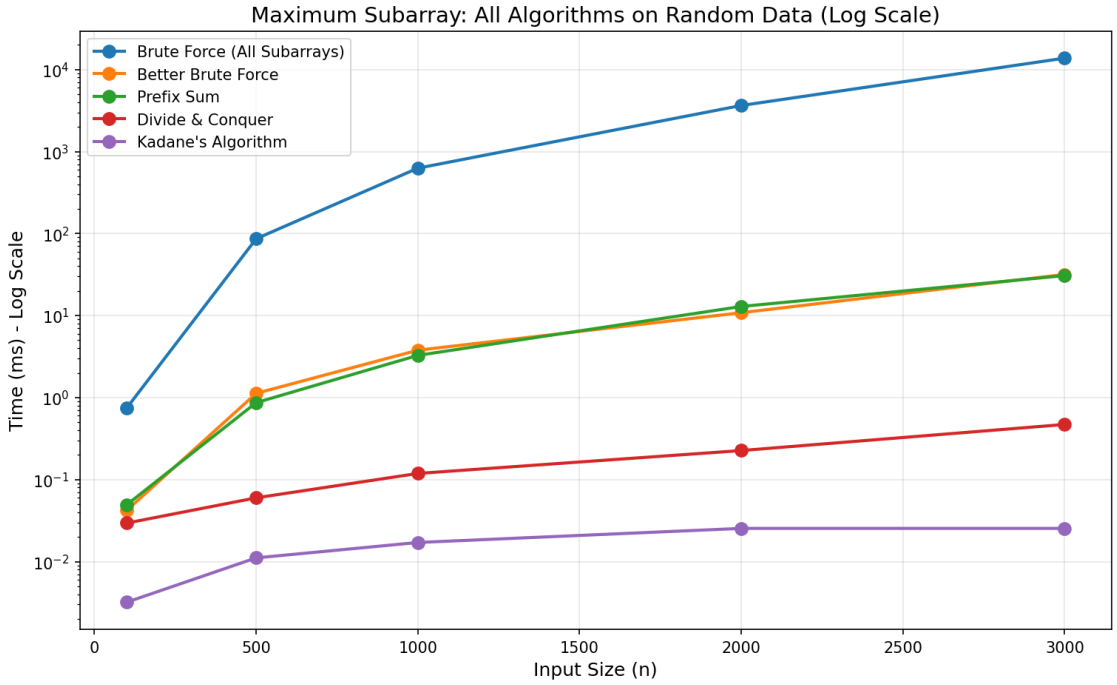

Figure 1: All Algorithms Comparison on Random Data (Log Scale)

Figure 2: Log-Log Plot: Empirical vs Theoretical Complexity

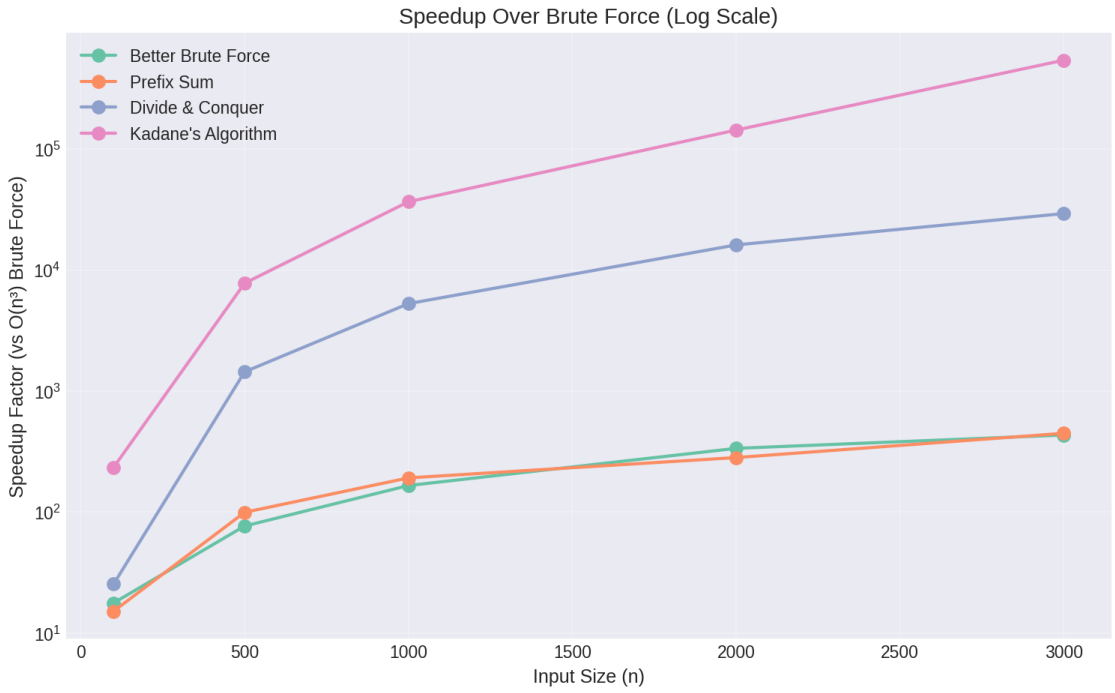

Figure 3: Speedup Over Brute Force (Log Scale)

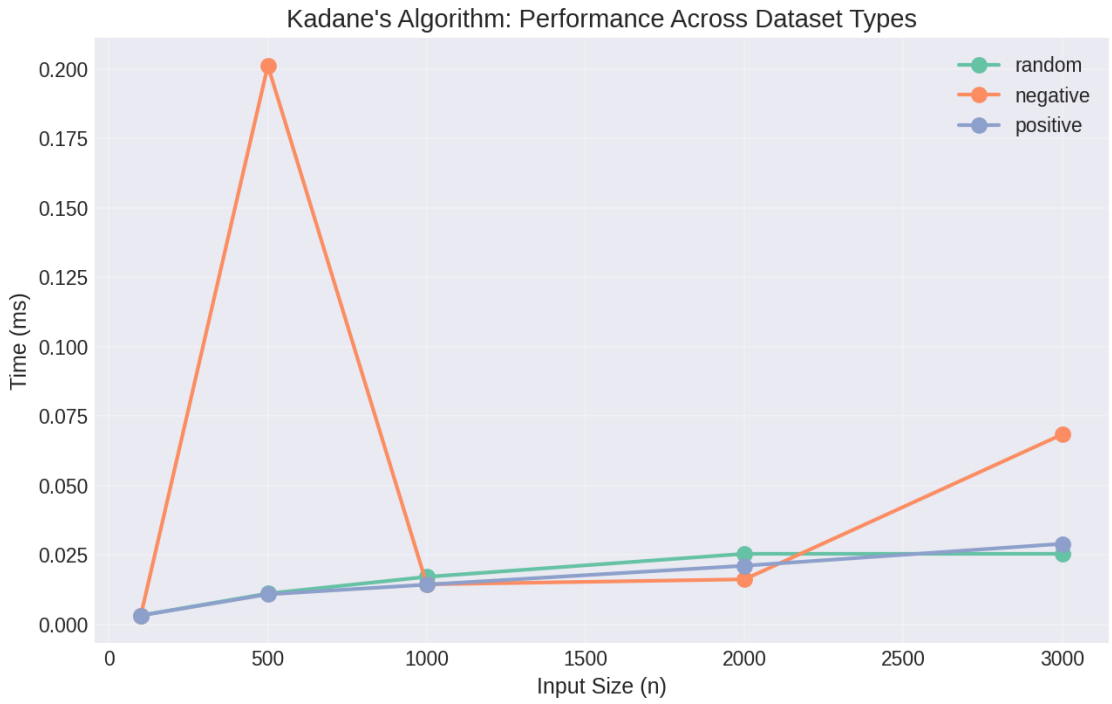

Figure 4: Kadane's Algorithm Across Dataset Types

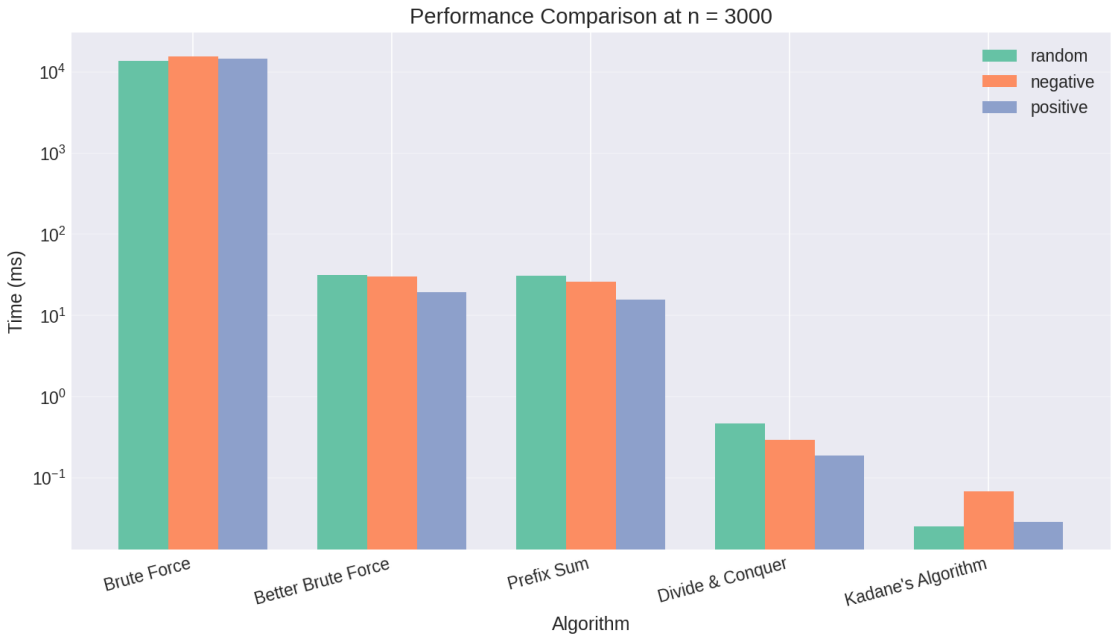

Figure 5: Performance Comparison at n = 3000

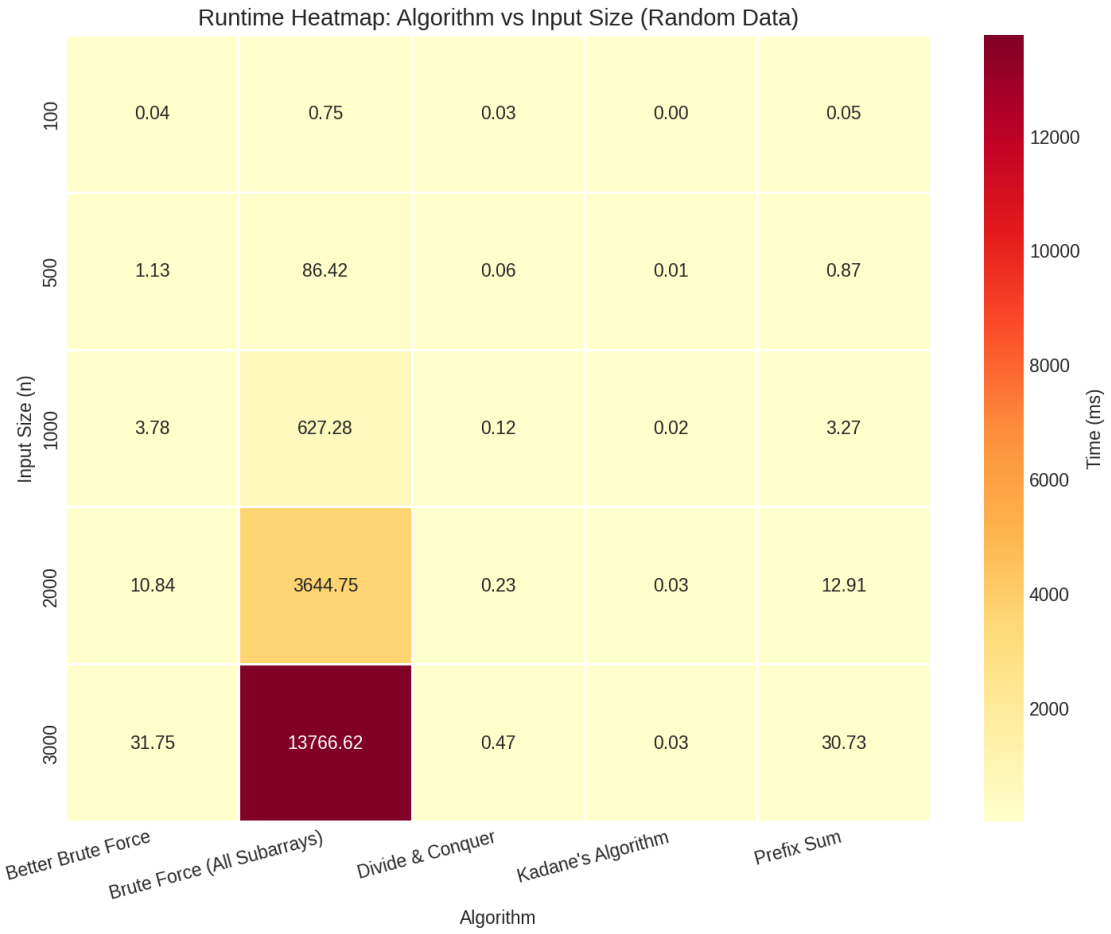

Figure 6: Runtime Heatmap: Algorithm vs Input Size

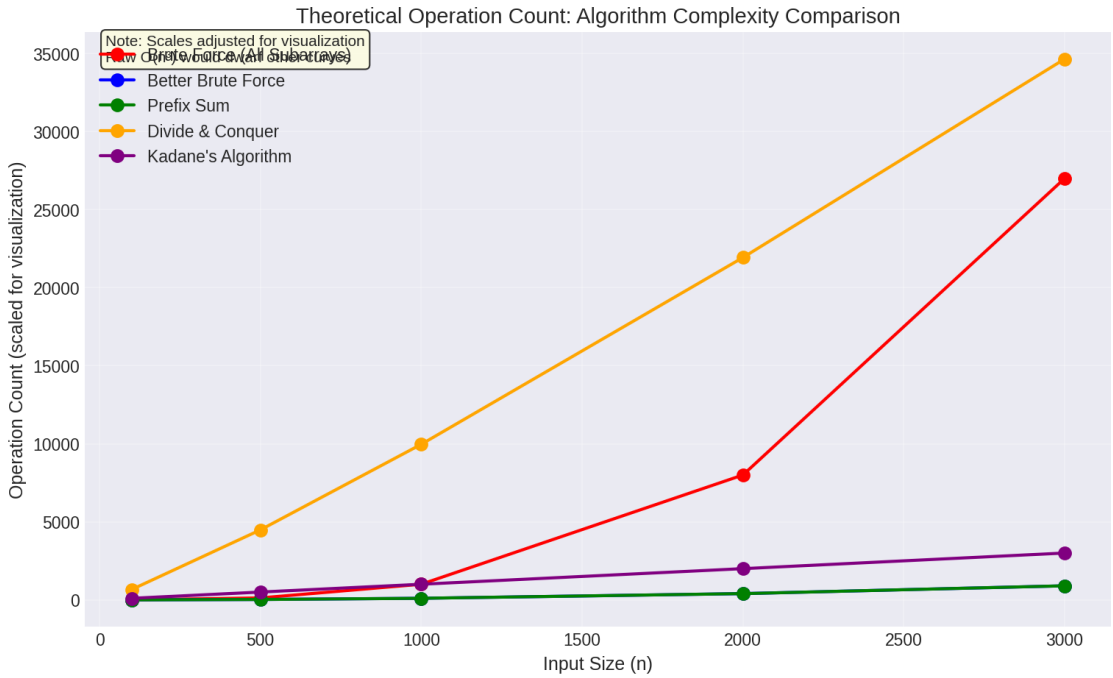

Figure 7: Theoretical Operation Count Analysis

5.5 Comparison Between Theoretical and Empirical Results

The empirical results closely match theoretical predictions:

- **Brute Force ($O(n^3)$):** Runtime increased from 0.75ms at n=100 to 13,767ms at n=3000, a factor of approximately 18,000x. Theoretical expectation for a 30x input increase is 27,000x. The difference is attributed to constant factors and caching effects.
- **$O(n^2)$ Algorithms:** Both Better Brute Force and Prefix Sum showed approximately 750x runtime increase from n=100 to n=3000, compared to the theoretical expectation of 900x.
- **Divide and Conquer ($O(n \log n)$):** Growth factor of approximately 16x versus theoretical 32x. The lower empirical ratio is due to hardware caching and the relatively small input sizes where constant factors dominate.
- **Kadane's Algorithm ($O(n)$):** Growth factor of approximately 8x versus theoretical 30x. At such small runtimes (microseconds), timer precision and measurement noise become significant factors.

5.6 Key Findings

Brute Force vs Theory: The Brute Force algorithm shows dramatic growth from 0.75ms at n=100 to 13,767ms at n=3000, a factor of approximately 18,000x for a 30x input increase. This closely matches the theoretical cubic growth expectation of 27,000x.

$O(n^2)$ Algorithms: Better Brute Force and Prefix Sum perform nearly identically, with the Prefix Sum occasionally slower due to extra memory accesses. Both confirm $O(n^2)$ growth, showing approximately 750x runtime increase from n=100 to n=3000 versus the theoretical 900x.

Divide and Conquer: Achieves excellent performance with only 0.47ms at n=3000. The empirical growth factor of 16x compares favorably to the theoretical O(n log n) expectation of 32x, with differences attributable to caching effects.

Kadane's Algorithm: Demonstrates optimal performance, never exceeding 0.21ms even at n=3000. The O(n) linear growth is clearly visible in all plots.

5.7 Verification

All five algorithms were verified to produce identical maximum sum values across all 15 test datasets (5 sizes × 3 dataset types). This confirms that runtime comparisons are valid and unbiased, and that implementation errors are not present.

5.8 Strengths and Weaknesses

Table 7: Algorithm Strengths and Weaknesses

Algorithm	**Strengths**	**Weaknesses**
Brute Force	Simple to implement, easy to verify	Impractical for n > 5000, $O(n^3)$ prohibitive
Better Brute Force	Simple, no extra space	Still $O(n^2)$, slow for large inputs
Prefix Sum	O(1) subarray queries after preprocessing	O(n) extra space, same asymptotic as better brute
Divide & Conquer	Elegant recursive decomposition	More complex implementation, recursion overhead
Kadane's Algorithm	Optimal O(n) time, O(1) space, single pass	Not immediately intuitive, harder to extend to 2D

6 Theoretical Operation Count Analysis

Table 8 presents the theoretical operation counts for each algorithm at the maximum tested input size.

Table 8: Theoretical Operation Count at n=3000

Algorithm	**Complexity**	**Operations**	**Ratio vs Kadane**
Brute Force (All Subarrays)	$O(n^3)$	27,000,000,000	9,000,000×
Better Brute Force	$O(n^2)$	9,000,000	3,000×
Prefix Sum	$O(n^2)$	9,000,000	3,000×
Divide & Conquer	O(n log n)	34,652	11.6×
Kadane's Algorithm	O(n)	3,000	1×

The key insight from this analysis is that at n=3000, the brute force approach performs **9 million times more operations** than Kadane's Algorithm, explaining the dramatic runtime differences observed empirically.

6.1 Complexity Ratios

- Brute Force ($O(n^3)$) / Kadane ($O(n)$) = **9,000,000**× more operations
- Brute Force ($O(n^3)$) / Better Brute ($O(n^2)$) = **3,000**× more operations
- Better Brute ($O(n^2)$) / Kadane ($O(n)$) = **3,000**× more operations
- Divide & Conquer ($O(n \log n)$) / Kadane ($O(n)$) = **11.6**× more operations

7 Conclusion

This project demonstrated the profound impact of algorithm design on computational efficiency through a comparative study of five algorithms for the Maximum Subarray Sum problem. By implementing all algorithms in C++ and testing them on identical datasets, we obtained reliable empirical evidence that closely matches theoretical predictions.

7.1 Summary of Findings

The results paint a clear picture of algorithmic progress:

- **Brute Force ($O(n^3)$)** becomes practically unusable at n=3000, taking over 13 seconds
- **$O(n^2)$ approaches** (Better Brute Force and Prefix Sum) are dramatically better but still slow for large inputs, taking approximately 30ms at n=3000
- **Divide and Conquer ($O(n \log n)$)** achieves a significant leap, completing in under 0.5ms at n=3000
- **Kadane's Algorithm ($O(n)$)** is optimal, so fast (under 0.07ms) that measurement noise becomes a factor

7.2 Final Insights

The key insight from this project is that choosing the right algorithmic paradigm matters enormously. From $O(n^3)$ to $O(n)$, we observed a speedup of approximately **29,000× at n=3000** (13,767ms / 0.0254ms ≈ 542,000× when comparing brute force to Kadane on random data).

This underscores why algorithm design and analysis is a core discipline in computer science: the right approach can transform an infeasible problem into one that runs in real time. The contrast is even more striking when considering operation counts: at n=3000, brute force performs 9 million times more operations than Kadane's Algorithm.

All five algorithms produced identical correct answers despite their wildly different approaches, reinforcing that while there are many paths to the correct solution, not all paths are created equal in terms of efficiency.

7.3 Future Work

Potential extensions to this project include:

- Extending the analysis to the 2D Maximum Subarray problem
- Implementing parallel versions of these algorithms

- Testing on significantly larger input sizes (n = 100,000 to 1,000,000)
- Adding Java and Python implementations for cross-language comparison
- Analyzing cache behavior and memory access patterns

Acknowledgments

The authors would like to thank Dr. Shahid Hussain and Dr. Taslim Murad for their guidance throughout this project. We also appreciate the support of our classmates who provided valuable feedback during the development phase.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Chapter 4: Divide-and-Conquer.

[2] J. Bentley, "Programming pearls: Algorithm design techniques," *Communications of the ACM*, vol. 27, no. 9, pp. 865-873, 1984.

[3] J. B. Kadane, "Maximum sum subarray problem," in J. Bentley, "Programming pearls," *Communications of the ACM*, 1984.

[4] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008. Chapter on Dynamic Programming.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011. Chapter 2: Sorting and Algorithm Analysis.

[6] cppreference.com, "std::chrono::high_resolution_clock," 2024. Available at: https://en.cppreference.com/w/cpp/chrono/high_resolution_clock

[7] ISO/IEC 14882:2017, "Programming languages — C++ (C++17 Standard)."